

Project Design Phase-II
Solution Requirements (Functional & Non-functional)

Date	12 July 2026
Project Name	CUSTOMER REGISTRY
Maximum Marks	4 Marks

Functional Requirements:

Following are the functional requirements of the proposed solution.

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	User Registration	Registration through form (full name, email, password) Admin creates agent accounts (role-assigned)
FR-2	User Authentication	Login via email & password JWT-based session token Role-based redirect to Customer / Agent / Admin dashboard Auto-provisioned admin login via configured credentials
FR-3	Complaint Management	Customer raises a complaint Customer tracks complaint status Admin assigns an agent to a complaint Agent updates complaint status Agent escalates a complaint to admin with a reason
FR-4	Communication	Real-time chat between customer and assigned agent Chat automatically disabled once complaint status is Closed
FR-5	Feedback & Ratings	Customer submits a rating and comments after complaint is resolved/closed Agent/Admin view submitted feedback on closed complaints
FR-6	Notifications	In-app notification generated on complaint status change Customer/agent can mark notifications as read
FR-7	Profile Management	User views their profile User updates name, phone, and address
FR-8	Customer History	Agent views a customer's full past complaint history for context
FR-9	Admin Analytics & Reporting	Admin views feedback analytics dashboard (average rating, rating distribution) Agent performance leaderboard

Non-functional Requirements:

Following are the non-functional requirements of the proposed solution.

FR No.	Non-Functional Requirement	Description
NFR-1	Usability	Clean, role-based dashboards for Customer, Agent, and Admin with minimal steps to raise, track, and resolve a complaint; responsive React UI with clear navigation.
NFR-2	Security	JWT-based authentication, bcrypt password hashing, role-based route protection (customer/agent/admin), and CORS restricted to only the deployed frontend origin.
NFR-3	Reliability	MongoDB Atlas managed database ensures consistent, durable storage of complaint, message, and feedback data across all user roles.
NFR-4	Performance	Lightweight REST API built on Express with JSON responses; frontend built with Vite for fast builds and optimized page loads.
NFR-5	Availability	Frontend hosted on Vercel and backend hosted on Render, both auto-redeploying on every push; note: the backend is currently on Render's free tier, which spins down after inactivity and can add ~50 seconds of delay to the first request.
NFR-6	Scalability	Modular MVC backend architecture allows new features (e.g., SMS/email notifications) to be added without restructuring existing code; MongoDB's flexible document model accommodates growing complaint and user volume.